

1. Introduction

Deep learning models, particularly in computer vision applications, are sensitive to variations in input data. Dataset augmentation has been shown to improve model generalization and resilience to perturbations (Cubuk et al., 2019). Randomized or automated augmentation strategies, such as AutoAugment and RandAugment, apply stochastic transformations to images, including geometric modifications, photometric adjustments, and noise injection (Cubuk et al., 2020). The Jetson Nano, a compact edge computing platform, offers CUDA-enabled GPU acceleration suitable for efficient image processing. This proposal describes a methodology to implement randomized image filtering on the Jetson Nano while ensuring compatibility with existing TensorRT-based inference pipelines, which cannot be modified due to audit requirements.

2. Proposed Methodology

2.1 Environment Setup

To ensure reproducibility and compatibility with the Jetson Nano, the framework will be developed and initially tested on an Ubuntu 18.04 system replicating the target device's software stack. This includes:

- Python 3.6.9
- OpenCV 4.5.0 with CUDA support
- NumPy and SciPy for auxiliary image operations

All library versions and installation commands will be documented for direct deployment on the Jetson Nano.

2.2 Randomized Filtering Pipeline

The framework will implement a **fully automated image augmentation pipeline**. The “how” is as follows:

1. Image Input:

- Images will be read from a user-specified folder.
- Metadata such as original resolution and filename will be recorded for traceability.

2. Filter Selection and Randomization:

- A catalog of filters will be defined, including:
 - **Photometric adjustments:** brightness, contrast, saturation, hue
 - **Geometric transformations:** rotation, scaling, flipping
 - **Noise injection:** Gaussian, salt-and-pepper
 - **Blur operations:** Gaussian, median, motion blur
- For each image, filters will be selected randomly using a pseudo-random number generator with a seed recorded in the metadata.
- Filter parameters (e.g., rotation angle, brightness factor) will also be randomized within defined ranges.

3. GPU-Accelerated Execution:

- OpenCV CUDA functions will be used wherever possible (e.g., `cv::cuda::GaussianBlur`, `cv::cuda::warpAffine`) to offload computations to the Jetson Nano GPU.
- Images will be processed in batches to maximize GPU utilization while avoiding memory overflow.

4. Output and Logging:

- Filtered images will be saved to a separate output folder.
- Each image will have a corresponding log file detailing: applied filters, parameter values, random seeds, and timestamps.
- This ensures reproducibility and auditability, which are essential for compliance with the existing project protocols.

2.3 Integration with Existing Systems

The pipeline will run **independently** of the TensorRT inference engine. Filtered images can be used for:

- Offline dataset augmentation for robustness testing
- Optional real-time preprocessing for live camera feeds, without altering the inference engine

No modifications to the existing engine will be necessary, preserving compliance with public-funded audit requirements.

2.4 Validation and Performance Assessment

Validation and performance assessment will be conducted as follows:

- **Functional Validation:** Process a small batch of images (e.g., 10–20) and verify that filters are applied correctly.
- **Performance Profiling:** Time the execution of larger batches (e.g., 150 images at 720p resolution) to evaluate GPU utilization, memory usage, and throughput.
- **Visual Inspection and Metadata Check:** Ensure that randomization is correctly logged and reproducible using the recorded seeds.

3. Expected Outcomes

- A GPU-accelerated framework for randomized image filtering on the Jetson Nano
- Automated dataset augmentation suitable for robustness testing of deep learning models
- Benchmark data on GPU performance and scalability
- Comprehensive documentation ensuring reproducibility and auditability

References

- [1] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "AutoAugment: Learning augmentation strategies from data," *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 113–123, doi: 10.1109/CVPR.2019.00021.
- [2] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical automated data augmentation with a reduced search space," *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2020, pp. 702–703, doi: 10.1109/CVPRW.2020.00089.
- [3] OpenCV, "Image Filtering - cv::cuda::createBoxFilter," *OpenCV Documentation*. [Online]. Available: https://docs.opencv.org/3.4/dc/d66/group__cudafilters.html. [Accessed: Sep. 27, 2025].